



Universidad Don Bosco

“Proyecto de Cátedra - Fase 1”

Asignatura

Desarrollo De Aplicaciones Con Software Propietario

Integrantes

- | | |
|--------------------------------------|----------|
| 1. Flores Figueroa, Josue Gamaliel | FF233029 |
| 2. Torres Reyes, Cristian Josué | TR240516 |
| 3. Ortiz Melendez, Michael Caleb | OM260275 |
| 4. Martinez Guerrero, Nathaly Ivania | MG260121 |
| 5. Peña Saravia, Jimmy Steeven | PS260123 |

Docente

Ing. Rene Mauricio Tejada

Actividad

Proyecto de Cátedra - Fase 1

Fecha de entrega: 24 de Agosto de 2026

3. Descripción general del proyecto

Sistema web inteligente para la gestión de renta de vehículos en pequeñas y medianas empresas de El Salvador

La propuesta consiste en desarrollar una aplicación web que centralice la administración de clientes, vehículos, reservas, contratos de alquiler, devoluciones, mantenimiento y reportes. El sistema estará orientado a pequeñas y medianas agencias salvadoreñas que actualmente pueden depender de hojas de cálculo, agendas, llamadas y mensajería para coordinar sus operaciones. La solución incorporará una función de inteligencia artificial que recomendará vehículos según las necesidades del cliente y únicamente dentro del inventario disponible.

3.1 Objetivo general

Desarrollar un sistema web seguro, modular y escalable para pequeñas y medianas empresas de renta de vehículos en El Salvador, que centralice la información de la flota y los clientes, automatice los procesos de reserva, entrega, devolución y mantenimiento, e incorpore un asistente inteligente para recomendar vehículos de acuerdo con las necesidades del usuario, con el propósito de reducir errores operativos, mejorar la trazabilidad y elevar la calidad del servicio.

3.2 Objetivos específicos

1. Centralizar la información de clientes, vehículos, categorías, tarifas, disponibilidad, reservas, alquileres, devoluciones y mantenimientos en una base de datos relacional.
2. Automatizar la consulta de disponibilidad y las reglas que impiden reservas superpuestas, así como el cálculo básico de tarifas, días de alquiler y cargos adicionales.
3. Integrar un servicio de inteligencia artificial que recomiende vehículos disponibles según cantidad de pasajeros, equipaje, tipo de viaje, transmisión, presupuesto y preferencias del cliente.
4. Implementar servicios web y controles de seguridad para autenticar usuarios, autorizar acciones por rol, validar datos y proteger la información personal y operativa.

5. Contenerizar la aplicación con Docker y proponer su despliegue en Microsoft Azure, manteniendo una arquitectura en capas que facilite las pruebas, el mantenimiento y la escalabilidad.

4. Gestión integral del proyecto

4.1 Análisis del problema y planteamiento de la solución

Contexto

Las micro y pequeñas empresas constituyen un componente relevante del tejido productivo salvadoreño. CONAMYPE informó que, durante siete años de gestión, brindó más de 276,000 servicios empresariales y alcanzó cerca de 52,000 registros MYPE, lo que evidencia la amplitud del sector atendido por la institución [1]. En el ámbito turístico, el directorio oficial de transporte de El Salvador incluye operadores registrados y arrendadoras de vehículos, por lo que la renta de automóviles forma parte de la oferta formal vinculada al turismo y la movilidad [2].

La observación inicial del equipo identifica además la presencia de agencias independientes que operan con pocos empleados y flotas reducidas. Esta afirmación se manejará como una hipótesis de trabajo y será validada mediante entrevistas, encuestas y observación durante la recolección de requisitos; no se asumirá como resultado definitivo sin evidencia de campo.

La necesidad de digitalización también es consistente con iniciativas nacionales recientes. En 2026, CONAMYPE, MITUR, KOICA y PNUD entregaron herramientas digitales a 200 MYPE del sector turismo para fortalecer sus capacidades tecnológicas, comerciales y de gestión [3]. Por tanto, una solución de gestión interna para pequeñas agencias se alinea con una necesidad real de transformación digital, siempre que sea sencilla, asequible y adaptable.

Problema identificado

Cuando la disponibilidad de los vehículos, las reservas y los contratos se administran en medios separados, la empresa carece de una única fuente confiable de información. Un cambio anotado en una agenda puede no reflejarse en una hoja de cálculo o en la conversación de otro empleado. Esto crea riesgos operativos que el estudio de campo deberá confirmar y dimensionar.

Los principales problemas que se pretende atender son los siguientes:

- Posibles reservas duplicadas o asignaciones de un mismo vehículo en períodos superpuestos.
- Demora para conocer qué vehículos están disponibles en un rango de fechas.
- Historial disperso de clientes, contratos, pagos, inspecciones, daños y cargos adicionales.
- Falta de alertas sobre devoluciones próximas, retrasos y mantenimiento preventivo.
- Dificultad para medir utilización de la flota, ingresos por período y vehículos más solicitados.
- Recomendaciones dependientes exclusivamente de la experiencia del empleado, sin criterios uniformes ni verificación automática de disponibilidad.

Usuarios afectados

- Propietarios o administradores, quienes necesitan indicadores y control general de la operación.
- Empleados de atención, responsables de registrar clientes, consultar disponibilidad y crear reservas.
- Encargados de flota, quienes registran entregas, devoluciones, inspecciones y mantenimientos.
- Clientes, quienes requieren información clara, opciones adecuadas y confirmaciones oportunas.

Impacto esperado del problema

Sin un registro centralizado puede aumentar el tiempo dedicado a buscar información, corregir inconsistencias y coordinar al personal. También se dificulta conocer el estado real de cada vehículo y reconstruir el historial de una operación ante una consulta o reclamo. El impacto exacto se medirá durante la investigación mediante tiempos de proceso, frecuencia de incidencias y percepción de los usuarios.

Solución propuesta

Se propone una aplicación web responsive con autenticación por roles. El sistema mantendrá una agenda central de disponibilidad, bloqueará conflictos de fechas, conservará el historial de cada vehículo y permitirá convertir una reserva en alquiler, registrar la devolución y programar mantenimiento. Los reportes ofrecerán indicadores básicos para apoyar decisiones operativas.

La función de IA actuará como asistente de recomendación. Primero, la lógica del sistema filtrará los vehículos realmente disponibles y compatibles con restricciones objetivas. Después, el servicio de IA ordenará o explicará las alternativas según las preferencias ingresadas. La recomendación será orientativa, mostrará los criterios utilizados y no confirmará una reserva sin aprobación del usuario.

Alcance y límites

El alcance inicial comprende una agencia con una o más sucursales configurables, pero prioriza el funcionamiento de una operación pequeña. No se incluirán en el primer prototipo una pasarela de pagos, rastreo GPS, integración con aseguradoras o instituciones públicas, aplicación móvil nativa, reconocimiento automático de daños ni fijación dinámica de precios. Estas capacidades podrán evaluarse como extensiones futuras.

Criterios de éxito propuestos

- Impedir reservas confirmadas que se superpongan para un mismo vehículo.
- Mantener trazabilidad desde la reserva hasta la devolución y el cierre del alquiler.
- Permitir consultar la disponibilidad mediante fechas y filtros desde una sola pantalla.
- Generar recomendaciones únicamente con vehículos disponibles y explicar por qué son apropiados.
- Obtener resultados satisfactorios en pruebas de usabilidad con usuarios representativos.

Los valores base y las metas cuantitativas definitivas se fijarán después de la recolección de datos; de esta manera se evitará declarar mejoras porcentuales sin una medición previa.

4.2 Diseño técnico inicial

Módulos principales

- Identidad y seguridad: inicio de sesión, recuperación de acceso, usuarios, roles y permisos.
- Clientes: datos de contacto, documentos requeridos, historial de reservas y alquileres.
- Flota: vehículos, categorías, características, tarifas, imágenes, kilometraje y estado.
- Reservas: consulta por fechas, validación de disponibilidad, creación, modificación y cancelación.

- Alquileres y devoluciones: contrato, entrega, inspección, combustible, kilometraje y cargos.
- Mantenimiento: servicios preventivos y correctivos, fechas, costos y bloqueo de disponibilidad.
- Reportes: utilización de la flota, ingresos, reservas, devoluciones pendientes y mantenimientos.
- Recomendador inteligente: captura de preferencias, filtrado de candidatos, explicación y registro de la recomendación.

Flujo de datos

El usuario interactúa con la interfaz web. La aplicación valida identidad y permisos, ejecuta las reglas de negocio y consulta los datos mediante Entity Framework Core. Para una recomendación, el sistema obtiene primero las opciones disponibles desde PostgreSQL y envía al servicio de IA solamente las características necesarias del viaje y de los vehículos candidatos. La respuesta se valida, se presenta como sugerencia y puede convertirse en una reserva mediante el flujo normal.

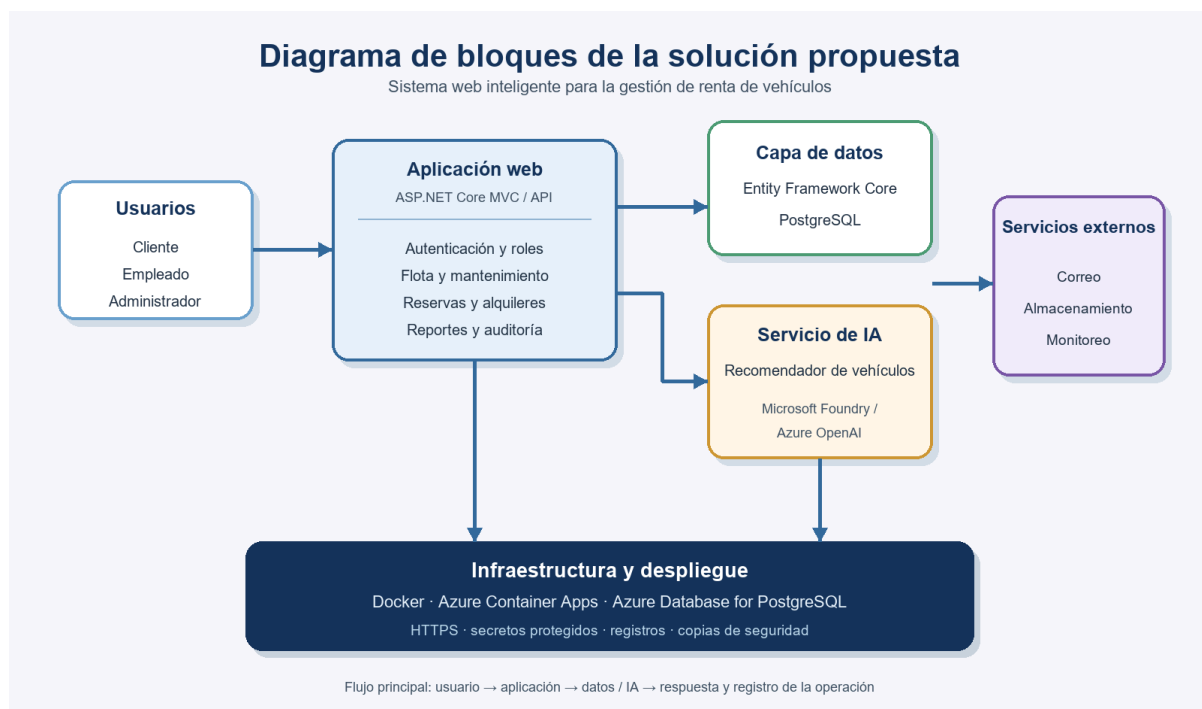


Figura 1. Diagrama de bloques y flujo principal de la solución propuesta.

Interacción entre componentes

La interfaz no accederá directamente a la base de datos. Las solicitudes pasarán por controladores o endpoints de ASP.NET Core, servicios de aplicación y reglas del dominio. La infraestructura implementará persistencia, correo, archivos, monitoreo y consumo del servicio de IA. Esta separación reduce el acoplamiento y permite probar la lógica sin depender permanentemente de servicios externos.

4.3 Estudio de factibilidad

Factibilidad técnica

El proyecto es técnicamente viable para un equipo de cinco integrantes porque utiliza tecnologías contempladas en la asignatura y con documentación oficial. ASP.NET Core permite crear aplicaciones y servicios web multiplataforma y orientados a la nube [5]. Entity Framework Core proporciona acceso relacional, consultas, seguimiento de cambios y migraciones para diferentes motores, incluido PostgreSQL mediante su proveedor correspondiente [6].

Docker permitirá ejecutar la aplicación y sus dependencias de manera consistente entre los equipos de desarrollo [7]. Para producción o demostración, Azure Container Apps admite despliegue de contenedores y escalado automático; en un perfil de consumo puede escalar a cero cuando no hay actividad [8]. La base de datos podrá alojarse en Azure Database for PostgreSQL Flexible Server, un servicio administrado con opciones de respaldo, mantenimiento y escalamiento [9].

Capacidades técnicas necesarias

- Programación en C#, ASP.NET Core MVC o Web App y diseño de APIs.
- Modelado relacional, SQL, Entity Framework Core y migraciones.
- HTML, CSS, JavaScript, accesibilidad y diseño responsive.
- Git y GitHub para control de versiones y revisión de cambios.
- Pruebas unitarias, de integración, seguridad y aceptación.
- Docker, configuración por ambiente, administración de secretos y fundamentos de Azure.
- Integración segura con un servicio de IA y evaluación de sus respuestas.

Riesgos técnicos y mitigación

- Curva de aprendizaje en Docker y Azure. Mitigación: crear una prueba de despliegue desde el inicio y documentar el procedimiento.
- Conflictos en el modelo de datos. Mitigación: acordar el diagrama entidad-relación y revisar migraciones antes de integrarlas.
- Indisponibilidad o costo del servicio de IA. Mitigación: aislarlo mediante una interfaz, limitar solicitudes y ofrecer una recomendación básica basada en reglas cuando el servicio no responda.
- Respuestas incorrectas de IA. Mitigación: filtrar candidatos en el servidor, exigir salida estructurada, validar identificadores y no permitir que la IA confirme operaciones.

Factibilidad económica

El desarrollo local puede realizarse con herramientas gratuitas o disponibles para uso académico: .NET, PostgreSQL, Git, GitHub y editores de código. La contenerización evita adquirir servidores durante la construcción. Los estudiantes elegibles pueden solicitar Azure for Students, que actualmente ofrece un crédito de USD 100 para doce meses y no exige tarjeta de crédito al registrarse [10].

Para un piloto, el gasto se concentrará en base de datos, almacenamiento, ejecución de contenedores y consumo de IA. El costo exacto dependerá de la región, capacidad, tráfico, retención de copias y cantidad de solicitudes; por ello deberá estimarse con la calculadora vigente de Azure antes del despliegue. El escalado a cero de la capa web puede reducir el consumo cuando el sistema está inactivo, aunque la base de datos administrada y otros recursos pueden mantener cargos [8], [9].

La conclusión económica es favorable para un prototipo académico y un piloto controlado. Para uso comercial se requerirá un presupuesto mensual, límites de consumo, alertas de costos y un plan de respaldo. No se asumirá que el crédito estudiantil constituye un modelo sostenible de producción.

Factibilidad operativa

La solución es operativamente viable si la interfaz reproduce el flujo cotidiano de la agencia y requiere poca capacitación. Se recomienda iniciar con una sucursal piloto, migrar datos esenciales, capacitar al personal y mantener un período breve de verificación paralela. Los

formularios deberán utilizar lenguaje sencillo, validaciones claras y estados visibles para cada vehículo.

La adopción dependerá de que propietarios y empleados participen en el levantamiento de requisitos y en las pruebas. Los beneficios esperados son una fuente única de información, menor exposición a conflictos de reserva, mayor trazabilidad y acceso a reportes. El riesgo principal es la resistencia al cambio; se mitigará con prototipos tempranos, demostraciones, manual breve y soporte durante el arranque.

Conclusión de factibilidad

El proyecto se considera factible en los ámbitos técnico, económico y operativo para la Fase II, condicionado a mantener el alcance priorizado, validar las necesidades con agencias reales y controlar los servicios de nube e IA.

5. Descripción técnica

5.2 Arquitectura del sistema

Enfoque arquitectónico

Se utilizará una arquitectura modular en capas dentro de una solución ASP.NET Core. Para el alcance académico se recomienda un monolito modular, ya que simplifica el despliegue y las transacciones sin impedir la separación lógica. No se propone una arquitectura de microservicios en la primera versión porque agregaría complejidad operativa innecesaria para una flota pequeña.

Capa de presentación

Aplicación ASP.NET Core MVC o Web App con diseño responsive. Presentará paneles diferentes para administrador y empleado, además de vistas para consulta y reserva. Los controladores recibirán solicitudes, validarán modelos y delegarán los casos de uso; no contendrán reglas de negocio complejas.

Capa de aplicación

Orquestrará los casos de uso: registrar vehículo, consultar disponibilidad, crear reserva, iniciar alquiler, cerrar devolución, programar mantenimiento, generar reporte y solicitar

recomendación. Definirá interfaces para persistencia, correo, archivos e IA, facilitando pruebas y sustitución de proveedores.

Capa de dominio

Contendrá entidades, objetos de valor y reglas centrales. Entre las entidades previstas se encuentran Usuario, Rol, Cliente, Sucursal, Vehículo, Categoría, Tarifa, Reserva, Alquiler, Inspección, CargoAdicional, Pago, Mantenimiento y Recomendación. Las reglas impedirán superposición de reservas, uso de vehículos fuera de servicio y cierre incompleto de devoluciones.

Capa de infraestructura

Implementará Entity Framework Core, PostgreSQL, repositorios cuando aporten valor, almacenamiento de imágenes, notificaciones, auditoría, registros y el adaptador del servicio de IA. Las credenciales se obtendrán desde configuración segura o secretos del entorno y nunca se almacenarán en el repositorio.

Aplicación web y servicios

ASP.NET Core será la plataforma de frontend y backend solicitada por la asignatura. La aplicación expondrá endpoints internos o una API web para los módulos que lo requieran. Las respuestas utilizarán modelos de transferencia de datos para evitar exponer directamente las entidades persistentes [5].

Base de datos y Entity Framework Core

PostgreSQL almacenará la información transaccional. Entity Framework Core administrará el mapeo, las relaciones, consultas y migraciones [6]. Se aplicarán restricciones e índices para proteger la integridad: matrícula o placa única, correo normalizado cuando corresponda, estados controlados y consultas eficientes por fecha, vehículo y estado.

Las operaciones de reserva deberán ejecutarse de forma transaccional. Antes de confirmar, el servidor consultará cruces de fechas y volverá a validar dentro de la operación para reducir condiciones de carrera. Las eliminaciones físicas se limitarán; los registros históricos utilizarán estados o baja lógica cuando sea necesario conservar trazabilidad.

Seguridad

Se propone ASP.NET Core Identity o un mecanismo equivalente para autenticación, contraseñas protegidas y gestión de roles. Se aplicarán autorización por políticas, HTTPS, protección antifalsificación en formularios, validación del lado del servidor, límites de solicitudes, registros de auditoría y manejo centralizado de errores.

El sistema tratará datos personales de clientes, por lo que deberá observar la Ley para la Protección de Datos Personales de El Salvador, Decreto Legislativo n.º 144. La ley regula la recolección, uso, procesamiento y almacenamiento legítimo e informado de datos personales [4]. El diseño deberá aplicar minimización, finalidad definida, control de acceso, conservación limitada y mecanismos para corregir o eliminar información cuando corresponda.

Inteligencia artificial

La IA recomendará vehículos, no aprobará clientes ni tomará decisiones legales o financieras. La entrada incluirá fechas, pasajeros, equipaje, presupuesto y preferencias, junto con una lista anonimizada de vehículos disponibles. No se enviarán documentos de identidad, licencias, direcciones ni datos de pago.

El backend validará que la respuesta haga referencia a identificadores permitidos y mostrará una explicación breve, alternativas y una advertencia de que la disponibilidad final debe confirmarse. Se registrarán versión del prompt, candidatos y resultado para pruebas. Microsoft recomienda un ciclo de identificar, medir, mitigar y operar los riesgos de sistemas generativos [11]; este enfoque se aplicará mediante casos de prueba, revisión humana, límites de alcance y monitoreo.

Contenerización y despliegue

La aplicación se empacará en una imagen Docker con compilación por etapas. En desarrollo se utilizará Docker Compose para la aplicación y PostgreSQL. La imagen no contendrá secretos y se ejecutará con configuración por ambiente [7].

La estrategia de Azure propuesta comprende Azure Container Registry para la imagen, Azure Container Apps para la aplicación, Azure Database for PostgreSQL Flexible Server para los datos y un servicio de almacenamiento para imágenes o documentos. El despliegue utilizará HTTPS, variables o secretos administrados, registros y alertas. El perfil de consumo de

Container Apps es apropiado para un piloto con tráfico variable porque admite escalado automático y a cero [8].

Respaldo, monitoreo y continuidad

Se habilitarán copias de seguridad de la base de datos, registros estructurados, métricas de errores y alertas de consumo. Antes de producción se probará la restauración, no solo la creación de respaldos. El servicio administrado de PostgreSQL reduce tareas de mantenimiento y ofrece capacidades integradas de respaldo y alta disponibilidad según la configuración elegida [9].

6. Plan de desarrollo

6.1 Recolección de requisitos

La recolección combinará evidencia documental y trabajo de campo. El objetivo es validar las hipótesis del problema, identificar variantes del proceso y priorizar funciones que una agencia pequeña realmente pueda adoptar.

Técnicas propuestas

- Entrevistas semiestructuradas con al menos tres propietarios o administradores de agencias y dos empleados de atención u operaciones.
- Encuesta breve a por lo menos quince clientes potenciales para conocer criterios de elección, canales utilizados y expectativas de confirmación.
- Observación, con autorización, del proceso de consulta, reserva, entrega y devolución en al menos una agencia.
- Revisión de formularios, contratos, hojas de cálculo o registros anonimizados que la empresa utilice actualmente.
- Taller How Might We con el equipo para convertir hallazgos en oportunidades de diseño y priorizarlas.

Pregunta guía How Might We

¿Cómo podríamos ayudar a una pequeña agencia salvadoreña de renta de vehículos a conocer su disponibilidad real, coordinar reservas y mantener trazabilidad de cada alquiler sin aumentar innecesariamente la carga administrativa?

Tratamiento de la información recolectada

Se solicitará consentimiento para participar, se evitará recopilar datos personales innecesarios y los ejemplos serán anonimizados. Los hallazgos se organizarán por usuario, tarea, problema, frecuencia e impacto. Después se construirán historias de usuario y criterios de aceptación; cualquier necesidad no validada quedará marcada como supuesto.

Requisitos funcionales preliminares

- RF-01. Autenticar usuarios y aplicar permisos para administrador y empleado.
- RF-02. Crear, consultar, actualizar y desactivar clientes.
- RF-03. Administrar categorías, tarifas, vehículos, fotografías y estados de flota.
- RF-04. Consultar disponibilidad por rango de fechas, categoría, capacidad y sucursal.
- RF-05. Crear, modificar y cancelar reservas sin permitir superposiciones confirmadas.
- RF-06. Convertir una reserva en alquiler y registrar contrato, entrega, kilometraje y combustible.
- RF-07. Registrar devolución, inspección, retrasos, daños informados y cargos adicionales.
- RF-08. Programar mantenimiento y retirar temporalmente vehículos de la disponibilidad.
- RF-09. Emitir confirmaciones y avisos mediante correo cuando el alcance del prototipo lo permita.
- RF-10. Generar reportes básicos de utilización, ingresos, reservas y mantenimiento.
- RF-11. Recomendar vehículos disponibles mediante IA y explicar los criterios de selección.
- RF-12. Conservar un registro de auditoría para operaciones sensibles.

Requisitos no funcionales preliminares

- RNF-01. Proteger todas las rutas privadas mediante autenticación y autorización por rol.
- RNF-02. Aplicar minimización, acceso restringido y tratamiento informado de datos personales conforme a la normativa salvadoreña [4].
- RNF-03. Responder las consultas habituales en un objetivo inicial de hasta tres segundos bajo la carga del piloto; el valor será validado mediante pruebas.
- RNF-04. Mantener respaldos y un procedimiento probado de restauración.

- RNF-05. Ofrecer interfaz responsive, navegación consistente y mensajes de error comprensibles.
- RNF-06. Mantener arquitectura en capas, convenciones de código, documentación y pruebas automatizadas.
- RNF-07. Ejecutarse de manera reproducible mediante contenedores Docker.
- RNF-08. Registrar errores, eventos críticos y consumo del servicio de IA sin almacenar información sensible innecesaria.

Priorización

Los requisitos se priorizarán con MoSCoW. Serán imprescindibles la autenticación, flota, clientes, disponibilidad, reservas, alquileres, devoluciones, mantenimiento básico y prototipo de IA. Las notificaciones avanzadas, pagos en línea, analítica predictiva y funciones móviles quedarán condicionadas al tiempo y a la validación del usuario.

6.2 Plan del prototipo funcional

El prototipo de la Fase II seguirá un desarrollo incremental. Primero se validarán los flujos en prototipos UX/UI; después se implementará una ruta completa desde la consulta de disponibilidad hasta la devolución. Cada incremento deberá quedar integrado, probado y demostrable.

Incremento 1 - Base técnica

Crear la solución ASP.NET Core, repositorio, arquitectura en capas, proyecto de pruebas, configuración de PostgreSQL, migración inicial y contenedores de desarrollo.

Incremento 2 - Administración de flota

Implementar autenticación, roles, CRUD de categorías, vehículos, clientes y estados operativos.

Incremento 3 - Reservas y alquileres

Implementar consulta por fechas, prevención de superposiciones, reservas, entrega y contrato de alquiler.

Incremento 4 - Devoluciones y mantenimiento

Registrar inspección de devolución, kilometraje, combustible, cargos y mantenimiento con bloqueo de disponibilidad.

Incremento 5 - IA, reportes y despliegue

Integrar el recomendador con candidatos previamente filtrados, crear reportes básicos, completar pruebas, generar la imagen Docker y desplegar un piloto en Azure.

Criterio de terminado

Una historia se considerará terminada cuando cumpla sus criterios de aceptación, tenga validaciones y autorización, incluya pruebas proporcionales al riesgo, esté revisada mediante control de versiones y funcione en el ambiente contenedorizado.

6.3 Alcance esperado del primer avance

- Estructura inicial de la solución ASP.NET Core y repositorio compartido.
- Arquitectura en capas definida con dependencias permitidas entre proyectos.
- Modelo de base de datos, diagrama entidad-relación y migración inicial.
- Entity Framework Core configurado con PostgreSQL.
- Autenticación básica y roles de administrador y empleado.
- CRUD principal de vehículos, categorías y clientes.
- Navegación básica entre panel, flota, clientes y reservas.
- Prototipos UX/UI validados para los flujos principales.
- Consulta preliminar de disponibilidad y regla contra reservas superpuestas.
- Diseño preliminar de la función de IA, contrato de entrada y salida, casos de prueba y estrategia de respaldo.
- Configuración inicial de Docker para reproducir el ambiente.

6.4 Entregables

- Documento del proyecto con planteamiento, factibilidad, arquitectura, plan, resultados y bibliografía.
- Diagramas de bloques, UML y entidad-relación.
- Diccionario de datos y reglas de integridad.
- Prototipos UX/UI y evidencia de validación.

- Modelo Canvas y presupuesto, desarrollados por los responsables de esas secciones.
- Repositorio con código fuente, historial de cambios e instrucciones de ejecución.
- Aplicación contenedorizada, pruebas y evidencia de despliegue piloto.
- Manual breve de usuario y documentación técnica.
- Presentación para exposición.

Cronograma de referencia para la Fase II

- Semanas 1 y 2: validar requisitos, prototipos, modelo de datos y arquitectura.
- Semana 3: preparar solución, seguridad, Entity Framework Core, PostgreSQL y Docker.
- Semana 4: implementar clientes, categorías, vehículos y estados de flota.
- Semana 5: implementar disponibilidad, reservas y alquileres.
- Semana 6: implementar devoluciones, mantenimiento y primera integración de IA.
- Semana 7: integrar reportes, pruebas, correcciones y despliegue en Azure.
- Semana 8: validar con usuarios, cerrar documentación y preparar la exposición.

Este cronograma es una propuesta relativa y deberá ajustarse a las fechas oficiales de la asignatura y a la disponibilidad del equipo.

7. Resultados esperados y conclusiones

7.1 Resultados esperados

Se espera obtener una aplicación funcional que permita a una agencia pequeña administrar su operación principal desde un único sistema. Los resultados son metas del proyecto y deberán comprobarse durante la Fase II; no representan beneficios ya medidos.

Resultados operativos esperados

- Disponibilidad centralizada y actualizada para reducir conflictos de asignación.
- Trazabilidad de cada vehículo desde la reserva hasta la devolución y el mantenimiento.
- Menor tiempo de búsqueda de información respecto de la línea base que se mida en campo.
- Registro uniforme de clientes, contratos, inspecciones y cargos.
- Reportes básicos para apoyar decisiones sobre utilización y mantenimiento de la flota.

- Recomendaciones coherentes con las preferencias y limitadas al inventario disponible.

Resultados técnicos esperados

- Solución ASP.NET Core con arquitectura en capas y responsabilidades separadas.
- Persistencia relacional mediante Entity Framework Core y PostgreSQL.
- Contenedores reproducibles para desarrollo y despliegue.
- Autenticación, autorización, validación, auditoría y protección de datos.
- Integración de IA aislada, evaluable y reemplazable.
- Despliegue piloto documentado en Microsoft Azure.

Indicadores propuestos para validación

- Cero reservas confirmadas superpuestas para un mismo vehículo en las pruebas de aceptación.
- Cien por ciento de los alquileres de prueba con historial de entrega y devolución.
- Al menos 80 % de participantes capaces de completar las tareas críticas sin ayuda durante la prueba de usabilidad; la muestra y el umbral deberán documentarse.
- Cien por ciento de recomendaciones de prueba limitadas a vehículos disponibles y existentes.
- Tiempo de consulta de disponibilidad y tasa de errores comparados con la línea base levantada en una agencia.

7.2 Conclusiones

El sistema de renta de vehículos constituye una propuesta académica realista porque combina gestión de información, automatización de procesos, servicios web, seguridad, base de datos, IA, Docker y despliegue en Azure dentro de un flujo empresarial coherente.

El valor principal no reside en crear un catálogo público, sino en ofrecer a pequeñas y medianas agencias una fuente única de información para coordinar la flota. La transformación digital promovida para las MYPE salvadoreñas y el reconocimiento institucional de operadores de transporte y arrendamiento respaldan la pertinencia del contexto elegido [2], [3].

Los retos más importantes serán validar las reglas reales de operación, mantener consistencia ante reservas concurrentes, proteger datos personales, contener costos de nube y evitar que la IA presente información no autorizada. La arquitectura propuesta mitiga estos riesgos

mediante reglas determinísticas, transacciones, control de acceso, separación por capas y validación de las respuestas de IA.

Se recomienda iniciar la Fase II con investigación de campo y un flujo vertical pequeño pero completo. Las funciones avanzadas deberán incorporarse solo después de que reservas, alquileres, devoluciones y mantenimiento funcionen de forma confiable. De esta manera, el equipo podrá entregar un prototipo demostrable sin perder control del alcance.

8. Bibliografía

- [1] Comisión Nacional de la Micro y Pequeña Empresa, “CONAMYPE celebra 35 años impulsando el desarrollo de las MYPE salvadoreñas y recibe reconocimientos por su trayectoria,” 25 de junio de 2026. [En línea]. Disponible: sitio oficial de CONAMYPE. [Consulta: 19 de agosto de 2026].
- [2] Ministerio de Turismo de El Salvador, “Transporte turístico,” El Salvador Travel. [En línea]. Disponible: directorio oficial de servicios turísticos. [Consulta: 19 de agosto de 2026].
- [3] Comisión Nacional de la Micro y Pequeña Empresa, “CONAMYPE, KOICA y PNUD fortalecen la transformación digital de 200 MYPE con Canastas Digitales ‘MYPE 360’,” 7 de julio de 2026. [En línea]. Disponible: sitio oficial de CONAMYPE. [Consulta: 19 de agosto de 2026].
- [4] Asamblea Legislativa de la República de El Salvador, Decreto Legislativo n.º 144, “Ley para la Protección de Datos Personales,” Diario Oficial n.º 219, tomo n.º 445, 15 de noviembre de 2024. [En línea]. Disponible: decreto oficial en PDF. [Consulta: 19 de agosto de 2026].
- [5] Microsoft, “ASP.NET documentation,” Microsoft Learn. [En línea]. Disponible: documentación oficial de ASP.NET Core. [Consulta: 19 de agosto de 2026].
- [6] Microsoft, “Entity Framework documentation hub,” Microsoft Learn. [En línea]. Disponible: documentación oficial de Entity Framework. [Consulta: 19 de agosto de 2026].
- [7] Docker Inc., “Introduction,” Docker Docs. [En línea]. Disponible: documentación oficial de Docker. [Consulta: 19 de agosto de 2026].
- [8] Microsoft, “Scaling in Azure Container Apps,” Microsoft Learn. [En línea]. Disponible: documentación oficial de escalado. [Consulta: 19 de agosto de 2026].

- [9] Microsoft, “What is Azure Database for PostgreSQL flexible server?,” Microsoft Learn. [En línea]. Disponible: documentación oficial del servicio. [Consulta: 19 de agosto de 2026].
- [10] Microsoft, “Azure for Students - Free Account Credit,” Microsoft Azure. [En línea]. Disponible: oferta oficial para estudiantes. [Consulta: 19 de agosto de 2026].
- [11] Microsoft, “Overview of Responsible AI practices for Azure OpenAI models,” Microsoft Learn. [En línea]. Disponible: guía oficial de IA responsable. [Consulta: 19 de agosto de 2026].